arXiv:2310.11703v2 [cs.DB] 16 Jun 2025

A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge

Le Ma* , Ran Zhang*, Yikun Han*, Shirui Yu, Zaitian Wang, Zhiyuan Ning, Jinghan Zhang, Ping Xu, Pengjiang Li, Wei Ju, Chong Chen, Dongjie Wang, Kunpeng Liu, Pengyang Wang, Pengfei Wang, Yanjie Fu, Chunjiang Liu †, Yuanchun Zhou, Chang-Tien Lu

***Abstract*—Vector databases (VDBs) have emerged to manage high-dimensional data that exceed the capabilities of traditional database management systems, and are now tightly integrated with large language models as well as widely applied in modern artificial intelligence systems. Although relatively few studies describe existing or introduce new vector database architectures, the core technologies underlying VDBs, such as approximate nearest neighbor search, have been extensively studied and are well documented in the literature. In this work, we present a comprehensive review of the relevant algorithms to provide a general understanding of this booming research area. Specifically, we first provide a review of storage and retrieval techniques in VDBs, with detailed design principles and technological evolution. Then, we conduct an in-depth comparison of several advanced VDB solutions with their strengths, limitations, and typical application scenarios. Finally, we also outline emerging opportunities for coupling VDBs with large language models, including open research problems and trends, such as novel indexing strategies. This survey aims to serve as a practical resource, enabling readers to quickly gain an overall understanding of the current knowledge landscape in this rapidly developing area.**

***Index Terms*—Vector Database, Retrieval, Storage, Large Language Models.**

I. Introduction

Vectors, particularly those in high-dimensional spaces, are mathematical representations of data, encoding the semantic and contextual information of entities such as text, images, audio, and video [1], [2]. These vectors are generally generated through some related machine learning models, and the generated vectors are usually high-dimensional and can be used for similarity comparison. The step of converting original unstructured data into vectors is the foundation of many artificial intelligence (AI) applications (including large language models (LLMs) [3], question-answering systems [4], [5], image recognition [6], recommendation systems [7], [8], etc.). However, in terms of managing and retrieving high-dimensional vector data, traditional databases designed for handling structured data are often inadequate. Vector databases, on the other hand, provide a specialized solution to these challenges.

Vector Databases (VDBs) are tools specifically designed to efficiently store and manage high-dimensional vectors. Specifically, VDBs store information as high-dimensional vectors, which are mathematical representations of data features or attributes [9]. Depending on the complexity and granularity of the underlying data, the dimensions of these high-dimensional vectors usually range from dozens to thousands. Unlike traditional relational databases, VDBs provide efficient mechanisms for large-scale storage, management, and search of high-dimensional vectors [10]–[12]. These mechanisms bring various efficient functions to VDBs, such as supporting semantic similarity search, efficiently managing large-scale data, and providing low-latency responses. These functions make VDBs increasingly integrated into AI-based applications.

VDBs have two core functions: vector storage and vector retrieval. The vector storage function relies on techniques such as quantization, compression, and distributed storage mechanisms to improve efficiency and scalability. The retrieval function of VDBs relies on specialized indexing techniques, including tree-based methods, hashing methods [13], graph-based models, and quantization-based techniques [14]. These indexing techniques optimize high-dimensional similarity search by reducing computational cost and improving search performance. In addition, hardware acceleration and cloud-based technologies have further enhanced the capabilities of VDBs, making them suitable for large-scale and real-time applications [15]–[17].

Therefore, compared with traditional databases, vector databases (VDBs) have three significant advantages: ***(1) Vector databases possess efficient and accurate vector retrieval capabilities.*** The core function of VDBs is to retrieve relevant vectors through vector similarity (distance), and this function is also the core of applications such as natural language processing (NLP), computer vision, and recommendation systems [18], [19]. In contrast, traditional databases can only query data based on exact matching or predefined conditions, and this kind of query method is relatively slow and often does not consider the semantic information of the data itself. ***(2) Vector databases support the storage and query of complex and unstructured data.*** VDBs can store and search data with high complexity and granularity, such as text, images, audio, video, etc. However, traditional databases, such as relational databases, are difficult to store this kind of data information well [20]. ***(3) Vector databases have high scalability and real-time processing capabilities.*** Traditional databases face challenges in effectively handling unstructured data and large volumes of real-time datasets [21]. However, VDBs can process vector data at large scale and in real time, which is crucial for modern data science and artificial intelligence applications [22]. By using technologies such as sharding [23],

* Equal contribution. † Corresponding author.

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

partitioning, caching, and replication, VDBs can distribute workloads and optimize resource utilization across multiple machines or clusters. Traditional databases, on the other hand, may face scalability bottlenecks, latency issues, or concurrency conflicts when handling big data [19].

Recent surveys on VDBs primarily cover fundamental concepts and practical applications of VDBs and vector database management systems. Some studies [12], [19], [24] focus on the workflow and technical challenges of VDBs, including key aspects such as query processing, optimization, and execution techniques. And some works [25] explore the critical role of VDBs in modern generative AI applications and provide an outlook on the future of VDBs. While these studies have their respective focuses, they do not provide a comprehensive survey of the overall storage and search technologies in VDBs, nor deliver a thorough analysis comparing the capabilities of existing VDBs. Furthermore, there is limited exploration of how these systems can integrate with rapidly advancing AI technologies, such as large language models (LLMs), to support modern data-intensive applications.

This gap shows the need for a comprehensive survey to consolidate current knowledge and uncover key research challenges related to VDB. To address this, our survey makes the following core contributions:

- We systematically review storage and retrieval techniques in VDBs, outlining their design principles and technological evolution.
- We provide a detailed comparative analysis of existing VDB solutions, highlighting their strengths, limitations, and typical application scenarios.
- We synthesize the main challenges, recent advancements, and future directions for VDBs, including open research problems and trends such as novel indexing strategies, adaptive query optimization, and integration with advanced AI capabilities.

This paper comprehensively summarizes the technologies related to vector databases, and systematically tests the performance of existing open-source vector databases. It also provides an outlook on the challenges that vector databases will face in the future. Through the summary of this paper, researchers can deepen their understanding of the field of vector databases. Figure 1 shows the overall framework of the paper, and we also construct a classification system of storage and search technologies for VDBs, as shown in Figure 2.

II. Storage

Efficient data management strategies are essential for the performance and scalability of VDBs. This section explores four key techniques: sharding and partitioning for data distribution, caching for reducing query latency, and replication for ensuring availability and fault-tolerance.

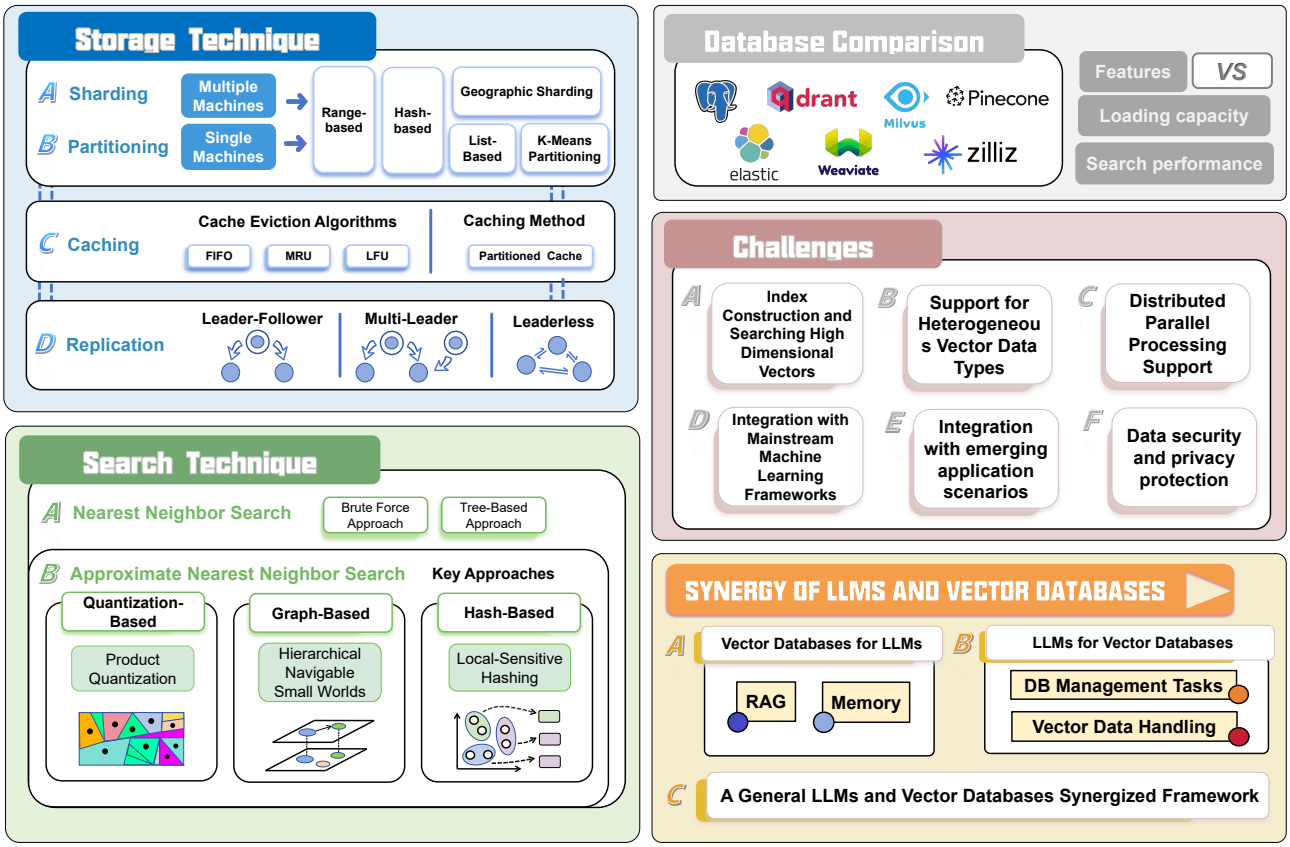

Fig. 1. Framework overview of this survey structure covering Storage Techniques, Search Techniques, Database Comparison, Challenges, and the Synergy of Large Language Models (LLMs) with VDBs. Each section represents a fundamental facet of the operation and integration of modern VDBs within advanced AI technologies.